

DetGuard — What It Is, What Was Done With It, Where It Could Go

Author: Rashi Bedse **Date:** 10 August 2026 **Status:** working document — supersedes the anomaly-analysis / `detguard lint` / DCI framing in `project-specification.md`

0. Why this document exists

The honours project was previously framed around `detguard lint` — a static+dynamic analyzer detecting "anomaly classes" in guardrail policies, scored by a new metric (DCI, Defense Concentration Index). That framing is being **retired as a built contribution** and demoted to a concept mentioned in future work.

Two reasons, both fatal on their own:

1. **Scope.** The full static pass (dead-by-logic, redundancy detection) was never implemented. Only the dynamic trace scan and a trivial `enabled: false` filter exist.
2. **Validity.** DetGuard is a self-built tool with three deployments, all authored by the same person who wrote the analyzer, the corpus, the mutations, *and* the policy being analyzed. A prevalence claim ("policies commonly contain shadowed rules") or a field-level critique ("existing guardrail evaluations measure the wrong thing") cannot be supported from $n=3$ self-authored artifacts. Measuring your own normalizer against your own mutations is circular.

This document records the ground truth of the system, what was actually done with the two banking applications, and an honest menu of directions with their validity constraints stated up front.

PART I — WHAT DETGUARD CURRENTLY IS

1. One-paragraph description

DetGuard is a policy-as-code enforcement engine for AI agent tool calls, plus an adversarial regression corpus that proves a policy still works after every change. Unlike most guardrail products, which filter input/output *text*, DetGuard checks the **tool call itself** — concrete arguments, deterministic conditions. Alpha, not published to PyPI, installed from source. Roughly 6,950 lines across 18 core modules, with a ~360-test suite that runs offline in 25–30 seconds.

Two things ship together and are frequently confused:

- an **enforcement library** (four hooks in `engine.py`), and
- a **corpus runner** (`runner.py` + `instantiate.py` + templates) that measures whether a policy actually holds against a role-parameterised attack corpus.

2. Architectural commitments

Core never imports an adapter or a framework. `engine.py`, `policy.py`, `registry.py`, `events.py` are pure — no

I/O, no framework types, testable without mocking. Enforced by convention, not by a lint rule.

No LLM sits in the enforcement path. Every condition is deterministic. The single exception, `llm_judge`, ships disabled and **fails open** — an unavailable judge must never silently become a block. This commitment extends to authoring time: policy derivation is mechanical, not model-assisted.

Determinism by construction. The corpus generator uses eight pure transform functions and no randomness, so the corpus is byte-identical across regenerations. CI asserts this explicitly.

3. The four hooks — the entire public contract

Hook	Fires on	Typical rules
<code>before_input</code>	user prompt, and separately on retrieved content with <code>is_retrieved=True</code>	<code>overt_injection</code> , <code>retrieved_instruction</code> , <code>retrieved_pii_redact</code>
<code>before_tool</code>	the decided tool call batch, before execution	<code>human_in_loop</code> , <code>unrequested_mutation</code> , <code>ungrounded_destination</code> , <code>external_destination_allowlist</code> , <code>call_budget</code> , <code>structuring</code> , <code>amount_bound</code>
<code>after_tool</code>	each tool result	<code>result_credential_leak</code> , <code>result_pii_redact</code>
<code>before_output</code>	the final answer	<code>output_credential_disclosure</code> , <code>output_pii_redact</code>

Each takes a `GuardContext` and a `PolicySet`, returns a `Verdict`.

4. Evaluation semantics

Three properties of `policy.evaluate()`, all deliberate:

- **S1 — exhaustive within a hook.** Every enabled rule bound to the hook runs; there is no short-circuit on first block. The full decision trace is the audit evidence, and "which layers *would* have caught this" is the defence-in-depth argument.
- **S2 — severity-ranked attribution.** When several rules fire, one is selected as the blocker: higher severity wins, and at equal severity a hard `block` beats a `require_hitl` pause. File order breaks remaining ties.
- **S3 — pipeline halts on block.** A block at an earlier hook means later hooks are never evaluated at all.

S1 and S2 together mean a rule can fire without ever being credited. S3 means a rule's hook can be unreachable in practice. (These two observations were the basis of the retired "novel anomaly classes" claim. They remain *true observations about the system*; what was overreaching was calling them a general taxonomy validated by evidence.)

5. The condition registry — 12 conditions

Contract, without exception: `fn(ctx, params) -> (bool, str)`. `True` means *the condition fired* (something the policy cares about was found), **not** "allow". What happens next is the rule's *action*, which is policy's business.

Condition	What it checks
<code>content_scan</code>	regex over hook text after normalisation; <code>applies_to</code> narrows to <code>any/retrieved/user</code>
<code>pii_detect</code>	detects sensitive values; detect only, never mutates
<code>pii_redact</code>	masks sensitive values and continues; composes with prior redactions rather than clobbering

sensitive_tool_call	fires if any decided call names a tool in the sensitive set — the HITL gate
tool_arg_matches	regex on one argument value of one tool
numeric_bound	numeric argument outside [min, max]; a non-parseable value also fires
call_budget	total call-count cap for one turn
repeated_call	same tool (optionally same args) past a threshold — the structuring check
ungrounded_arg	argument value appears nowhere in the user's own request — destination substitution
unrequested_tool	a mutating call the user's request did not licence — overreach, not attack
external_destination	data leaving for a destination outside the allowlist
llm_judge	optional caller-supplied judge; disabled, fails open

Actions: block, redact, require_hitl, warn, limit, notify. **Blocking actions:** block and require_hitl — both stop unattended execution.

6. Normalisation — and its current limits

registry.normalize() runs before every regex match. It does two things:

- strips **7** invisible codepoints (zero-width space / non-joiner / joiner, LTR and RTL marks, BOM, word joiner)
- folds **11** Cyrillic-and-friends homoglyphs back to Latin

Plus _haystacks() produces a second, whitespace-stripped form matched with a whitespace-optional rewrite of the pattern, because whitespace_pad inserts separators *inside* tokens where collapsing runs cannot see them.

Known gap, worth stating plainly: the homoglyph table is 11 hand-picked entries. Unicode's UTS #39 confusables.txt has thousands. There is no NFKC pass. Three of the eight corpus mutations have no normalisation defence at all (see §8).

7. The role vocabulary — 9 closed roles

Tool *names* mean nothing to a generic engine; refund_order is just a string. Roles are what templates, policy defaults and compliance mapping all key off. The vocabulary is **closed** — a role outside the list is a hard load-time error.

Role	Meaning	Gated by default
read_internal	reads trusted internal state	
read_untrusted	pulls in attacker-authorable content	
mutate_state	changes non-critical state	
mutate_identity	changes who/where the principal is	✓
move_value	money, goods, entitlements	✓
change_credential	auth material	✓
external_send	data leaves the perimeter	✓
external_fetch	retrieves from an attacker-influencable address	
destructive	irreversible	✓

A client tunes *down* from the gated set — a deliberate, logged decision — rather than tuning up from nothing.

8. Corpus generation — 16 templates × 8 mutations

`instantiate.py` binds templates to a client's `manifest.yaml` + `roles.yaml` **by role matching, not tool names** — a template requiring `move_value` binds to whatever tool in that manifest carries the role. A template that cannot bind is recorded in `_skipped.yaml` with a reason, never silently dropped.

16 templates: direct injection, indirect injection, record-field injection, obfuscated injection, markup concealment, destination substitution, context poisoning, tool chaining, claimed authority, unrequested mutation, exfiltration pretext, result leak, credential disclosure, malicious URL, simulation framing, threshold evasion.

8 mutations, all pure functions, and their current defence status:

Mutation	Kind	Neutralised by
<code>zero_width</code>	syntactic	<code>normalize()</code> invisible-char table
<code>homoglyph</code>	syntactic	<code>normalize()</code> — 11 entries only
<code>case_scramble</code>	syntactic	<code>re.IGNORECASE</code>
<code>whitespace_pad</code>	syntactic	<code>_despace()</code> + <code>_whitespace_optional()</code>
<code>comment_wrap</code>	structural	nothing
<code>base64_wrap</code>	encoding	nothing
<code>position_shift</code>	placement	n/a — tests attention, not matching
<code>politeness_wrap</code>	semantic	nothing — and normalisation structurally cannot help

`politeness_wrap` is the interesting one: it rewrites imperative mood into declarative ("please send X to Y" → "X is to be directed to Y"). Every other mutation changes bytes; this one changes *grammatical mood*, which is what "does this text contain an instruction?" heuristics actually detect.

9. prevented VS detected — the most important distinction in the runner

- **prevented** — the adapter exposes `set_tool_guard`, so `before_tool` is consulted inside the agent loop immediately before each tool body runs. A block stops the call: the transfer does not happen. Only `OpenAIAdaptersAdapter` offers this today, via the SDK's own `tool_input_guardrails`.
- **detected** — no such seam (`LangGraphAdapter`, `GenericAdapter`), so hooks are evaluated after `invoke()` returns. The trace is accurate but the side effect already happened. `blocked` here means "a live integration would have stopped this," not "did."

Every `results.json` records which mode a run used, per attack and in the summary. Averaging over both is how a benchmark comes to describe itself as a guardrail.

Non-obvious trap: a run's `enforcement` field reflects the *runner's own* measurement guard attaching to the agent's tools — independent of whether the target app's production loop calls DetGuard at all. An app with zero guardrail wiring can still show `enforcement: prevented` when *measured*. Do not conflate "this corpus run measured well" with "this app is protected in production." (`bankingass` is exactly this case — see §12.)

10. Scoring invariants

Each one was a real bug in a predecessor project. These are, arguably, the most transferable knowledge in the

structuring	before_tool	block	high	✓
amount_bound	before_tool	require_hitl	high	✗
llm_judge_intent	before_tool	require_hitl	critical	✗
result_credential_leak	after_tool	block	critical	✓
result_pii_redact	after_tool	redact	medium	✓
output_credential_disclosure	before_output	block	critical	✓
output_pii_redact	before_output	redact	medium	✓

PART II — WHAT WAS DONE WITH THE BANKING APPLICATIONS

13. The two applications

Both are the same underlying mini banking assistant, on different branches of `rashibedse/mini_banking_assistant`. Five tools, SQLite state, Groq-backed agent.

	D : /VSCODE/ bankingass	D : /VSCODE/bankingass_2
Branch	main	fix/policy-and-corpus (currently on demo/pr2-regression)
guardrail.py	absent	present
Production enforcement	none	@protect decorator on all five tools
detguard_adapter.py	present	present
config/ (manifest, roles, arg_hints, policy)	present	present
corpus/ (25 attacks + baseline)	present	present
Purpose	the genuinely-unguarded control	the guarded subject

This pair is the control/treatment design, and it is the strongest methodological thing in the project: a guardrail number measured against an agent that wouldn't have complied anyway is meaningless. CI asserts the unguarded fixture is genuinely vulnerable — defence rate must be 0% with `--guardrail off`.

Note `bankingass` has an adapter, config and corpus but no runtime guardrail. It is the concrete instance of the \$9 trap: it can be *measured* in `prevented` mode while having zero protection in its actual chat loop.

14. Role classification used

Identical in both apps — five tools, four roles:

```
roles = {
    search_customers: read_internal,
    get_account: read_internal,
    get_transactions: read_internal, read_untrusted, # the untrusted-source carrier
```

```
transfer_funds, move_value
update_credentials, update_credentials
```

Five of nine roles are unused: `mutate_state`, `change_credential`, `external_send`, `external_fetch`, `destructive`. This is a real limitation of the case study and it shows up directly in corpus binding (§15).

15. Corpus actually generated — 25 attacks from 16 templates

Six of sixteen templates **could not bind** and are recorded in `_skipped.yaml` with reasons:

Template	Reason
TPL-02 context/indirect	no tool with role <code>mutate_state</code>
TPL-07 context poisoning	no tool with role <code>mutate_state</code>
TPL-10 unrequested mutation	no tool with role <code>mutate_state</code>
TPL-11 exfiltration pretext	no tool with role <code>external_send</code>
TPL-13 credential disclosure	no tool with role <code>change_credential</code>
TPL-14 malicious URL	no tool with role <code>external_fetch</code>

Four mutation variants were also skipped — `politeness_wrap` was a **no-op** on TPL-03, TPL-06 and TPL-08 (those payloads are already declarative, which is precisely the point of those templates), and `position_shift` needs a payload separable from its carrier which TPL-15 does not have.

The 25 that generated span TPL-01, 03, 04, 05, 06, 08, 09, 12, 15, 16 with `case_scramble`, `homoglyph`, `whitespace_pad`, `zero_width`, `position_shift`, `base64_wrap`, `comment_wrap` and `politeness_wrap` variants.

16. Measured result (`results-on.json`, `guarded`, `bankingass_2`)

```
{
  "total": 25,
  "enforcement": "prevented",
  "defended": 23,
  "defense_rate": 0.92,
  "blocked": 23,
  "containment_rate": 0.92,
  "contained": 23,
  "coverage": 0.92,
  "succeeded": 0,
  "not_complied": 0,
  "inconclusive": 0,
  "mitigated": 0,
  "requires_approval": 0,
  "adapter_errors": 2,
  "skipped": 0
}
```

Read this carefully:

- **Zero attacks succeeded.** Nothing got through.
- **Defence rate 0.92, not 1.00** — because two runs hit `adapter_error` (Groq free-tier 429 rate limiting), which correctly drags coverage down rather than being silently counted as a pass. The scoring refuses to certify what it did not observe.
- **requires_approval: 0.** Across 25 attacks, the `human_in_loop` rule — severity `critical`, `require_hitl`, guarding money movement and identity change — **never once determined an outcome**. Earlier hooks and higher-ranked blockers always resolved first.

17. What is actually wired in the guarded app's live loop

From DEMO.md, and worth being precise about because the policy file contains rules this app cannot reach:

Hook	Wired where	Rules that actually run
before_input	main.py, user prompt	overt_injection
before_tool	@protect, all five tools	human_in_loop, unrequested_mutation, ungrounded_destination, external_destination_allowlist, call_budget, structuring
after_tool	@protect, all five tools	result_credential_leak, result_pii_redact
before_output	main.py, final answer	output_credential_disclosure, output_pii_redact

retrieved_instruction and retrieved_pii_redact are defined in the policy and **do not run in the chat loop** — they key off before_input(..., is_retrieved=True), and this app has no step that pulls a document into context before the turn. The corpus runner reaches them because it plants payloads through the adapter, which is why detguard run exercises more of the policy than the CLI does.

18. The live demo

Two scripted attacks, both verbatim from the corpus:

- 1. **Direct injection** (TPL-01) — on main the agent calls transfer_funds and CHK-1001 drops from 84,250.50 to 76,750.50. On the guarded branch overt_injection matches at before_input, so the agent is never invoked at all: no model call, no tool call, no database write.
- 2. **Indirect credential leak** (TPL-12) — a password planted in a transaction description. The user's request is entirely benign. On main the agent prints KX9T2MQ7VB4N. On the guarded branch result_credential_leak fires at after_tool and the real description never reaches the model.

The second one is the better demo and needs narration — there is no [blocked] banner, the turn completes normally, and the interesting thing is what is missing from the answer.

19. The regression demo

Branch demo/pr2-regression deliberately disables the overt_injection rule to show baseline.compare() catching it as a NEW_BREACH and failing CI. This is the "prove it still works after every change" story made concrete.

20. The real bug found and fixed

call_budget / repeated_call (the structuring conditions) were **structurally unable to fire under real pre-execution enforcement**. Both reason over a decided batch — a running total, a per-tool repeat count — but a prevented run intercepts one call at a time as the agent decides it. Checking each call against a fresh one-element list means every call is, on its own, within every limit.

Fixed by threading a same-turn calls accumulator through **both** implementations — a plain list in runner.py's _tool_guard, a _TURN_CALLS contextvar in bankingass_2's guardrail.py mirroring the existing _TURN_PROMPT pattern. Verified safe by hand-tracing that all other per-call-scoped rules are monotonic and produce identical results whether evaluated singly or as part of a growing batch.

No test in the suite exercised this scenario before the fix. This is a genuine existence-proof finding: a rule that reads as working, loads without error, appears in the policy, and cannot fire.

21. Also found, not fixed

`require_hitl` does not actually pause or queue for approval. It refuses the call exactly like `block`, just labelled `approval_required` instead of `blocked`. No HITL resume mechanism exists anywhere in the codebase. Worth stating precisely if it comes up — not implying more than that.

PART III — WHAT CLAIMS THE EVIDENCE SUPPORTS

22. The validity constraint, stated once

Validity here is not determined by sample size. It is determined by **claim type**.

Claims that survive n=3 self-authored:

- **Existence** — "this failure mode is possible; here is a concrete instance." n=1 is sufficient by construction. The §20 structuring bug and the §16 `requires_approval: 0` observation are both this shape. You never need prevalence to say *this can happen and nothing catches it*.
- **Engineering / design** — "I built this, here is why each decision exists, here is a demonstration." The claim is about the artifact. Scoped as a case study.
- **Conformance to an external yardstick** — measured against something you did not author.

Claims that do not survive:

- **Prevalence** — "X% of policies have this problem."
- **Field-level critique** — "existing guardrail evaluations measure the wrong thing."

Both need data that does not exist here.

23. The circularity trap, specifically

Any measurement of the form "*extending `normalize()` reduced evasion by X% on the corpus*" is circular: the same person wrote the normaliser, the mutations, and the templates. The adversary is not independent.

The fix is to change the yardstick, not the experiment. Measuring homoglyph coverage against Unicode UTS #39 `confusables.txt` — a table published and maintained by the Unicode Consortium — is a *conformance* claim against an external, independently-authored standard. "Coverage went from 11 entries to the full table" is true regardless of who uses DetGuard or how many deployments exist. The moment you append "...and this reduced evasion by X% on my corpus," you are back to circular.

24. External-data feasibility — what was checked, 10 Aug 2026

The idea was to point DetGuard's role taxonomy at tools nobody here wrote. Findings:

- **Official MCP registry carries no tool schemas.** `registry.modelcontextprotocol.io/v0/servers` returns `name, description, title, version, remotes, repository`. No tool names, no input schemas. Getting actual tool definitions requires connecting to each server or reading its source.
- **Glama indexes ~70,000 servers** but exposes no documented free public API; the tool-schema access advertised elsewhere is via a paid third-party scraper.
- **The descriptive survey is already done, at scale, by a credible institution.** Stein, M. (2026), *How are AI*

agents used? Evidence from 177,000 MCP tools, UK AI Security Institute / University of Oxford. arXiv:2603.23802. 177,436 public MCP tools from GitHub and Smithery, Nov 2024–Feb 2026. Dataset available on request, not publicly released.

Do not attempt to out-scale this. The prevalence study is gone.

25. Why the Stein paper is an asset, not a competitor

Read where it stops.

Its taxonomy is **perception / reasoning / action**, plus generality and task domain via the O*NET occupational framework. That is an *economic consequentiality* classification, built to answer "what work are agents doing and how high-stakes is it." It is not an enforcement vocabulary — nothing in it tells you what rule to write.

Its findings hand over the problem statement from a source no examiner will challenge:

- Action tools grew from **27% to 65%** of total tool usage over 16 months
- Financial transaction tools called out as a fast-growing high-stakes category
- 28% of MCP servers show AI-assistance in creation, rising to **62% of new servers by Feb 2026**
- Closing move: governments and regulators can use this monitoring method to extend oversight *beyond model outputs to the tool layer*

That last point is the seminar's governance question, asked by UK AISI. And the paper **stops at monitoring**. It observes the tool layer; it never enforces on it. It states explicitly that a large action space does not itself constitute harm — it merely permits a wide range of actions. There is no bridge from "we classified 177k tools" to "therefore this tool call is blocked."

That bridge is what `roles.py` + `derive_policy()` already are. Same input — a tool classification — different output. Stein classifies to inform regulators; DetGuard classifies to generate enforcement.

26. The governance reframe

The presentation feedback was: *policies cannot just be human-generated, they need approval by some official body.*

The answer already exists in the codebase and is the **opposite** of "get each policy approved":

You do not standardise policies. You standardise the role taxonomy, and policies derive from it mechanically.

A body publishing nine tool-role classes is plausible. A body approving every company's individual YAML file is not. Under this model the human's only job is classifying *their own* tools into someone else's published vocabulary — a much smaller, much more auditable act than authoring rules. `derive_policy()` then produces the policy by rule, with `unfilled()` naming exactly what still needs a human and why.

This reframe costs nothing to make and converts the objection into the contribution.

PART IV — DIRECTIONS

Each entry states the claim type (§22) so validity is visible up front.

27. Tier 1 — recommended, low risk

27.1 Taxonomy coverage on external tools (*conformance / instrument-validity*)

The threat this addresses is real and currently unanswered: the nine roles were designed while looking at banking assistants. Is that a general vocabulary, or a curve fitted to three apps? Right now that question cannot be answered, and it is the first thing a sharp examiner will ask. §14 makes it worse — the case study exercises only four of the nine roles.

Do: take 50–150 MCP servers nobody here wrote, pull tool schemas from their GitHub source, classify every tool into the nine roles. Report what fraction classify cleanly, what fraction need a role that does not exist, and where classification is ambiguous. Compare against Stein's action/perception/reasoning split on overlapping servers.

This is a coverage-of-vocabulary claim, not prevalence. It survives small n because it is about the instrument, not the ecosystem. It is falsifiable — if 40% of real tools do not fit, that is a genuine negative result worth reporting.

Unverified: per-server cost of extracting schemas. Some servers declare tools in a clean manifest; others bury them in code. Test-harvest a handful before committing.

27.2 Derivation coverage (*engineering + measurement on own artifact*)

Do: measure what fraction of a working policy follows mechanically from a role classification. `unfilled()` already computes the gap; `derive_policy()` already documents the two params it deliberately refuses to fill and why. Instrument across both banking apps plus synthetic manifests exercising the five currently-unused roles.

Nearly free. Directly answers the governance question. Low ceiling but zero risk.

27.3 Harness invariants as an experience report (*design knowledge*)

Do: write up §10 and §20 properly. Non-obvious correctness invariants for agent guardrail *evaluation harnesses*, each discovered through a real bug, each now enforced in code with the reasoning in a comment at the point of enforcement.

Anyone building such a harness hits these, and they are currently undocumented in the literature. Transferable knowledge that does not need $n > 3$ — this is a patterns / experience-report contribution, a recognised genre.

Pair it with the **instrument-validation** point: CI asserts the unguarded fixture is genuinely vulnerable (0% defence with `--guardrail off`). State it as a methodological requirement for anyone measuring a guardrail; do not claim others fail it.

28. Tier 2 — real work, real payoff, some risk

28.1 Unicode conformance for normalisation (*conformance*)

Extend `normalize()` from 11 hand-picked homoglyphs to full UTS #39 `confusables.txt`, add an NFKC pass, and add handling for the two undefended non-semantic mutations (`base64_wrap` decoding, `comment_wrap` markup stripping).

Report **coverage against the published Unicode table**, not evasion rate on the corpus (§23). Clean, external, unfalsifiable-by-obscurity.

Risk: low. Mostly mechanical. Watch for false positives — aggressive NFKC folding can make distinct legitimate strings collide.

28.2 Deterministic mood detection (*engineering, may fail*)

`politeness_wrap` defeats every current defence by changing grammatical mood rather than bytes, and normalisation

is structurally incapable of helping. Add a 13th condition: rule-based imperative/directive-mood detection, POS-tagged, **no model** — an LLM here breaks DetGuard's core claim.

Highest novelty ceiling on this list and the most likely to not work. State that as a risk, not a rhetorical hedge. Even a negative result is reportable: "deterministic mood detection is insufficient for semantic evasion; this is where the no-LLM commitment has a real cost."

28.3 Real human-in-the-loop (*engineering*)

Build genuine pause/resume so `require_hit1` stops being a relabelled block (§21). Then measure escalation burden — approvals per 100 turns, and how often escalation is warranted. Connects to alert-fatigue literature.

Largest engineering item here. Also the one that would make `containment_rate` mean what it says.

29. Tier 3 — noted, not recommended now

- **Benign-traffic false positive study.** Genuinely important — a 100% defence rate is trivially achievable by blocking everything, and `baseline.py` already treats `BENIGN_BLOCKED` as build-failing. But building a credible benign corpus is a project in itself, and measuring your own policy against your own benign set has the §23 circularity problem.
- **Layer ablation.** Enable/disable each layer, measure marginal contribution via the existing `--enable-layer` machinery. Gets the same headline as DCI with standard, unchallengeable methodology. Cheap — but it is the retired contribution minus its distinctiveness, and it is still measurement-on-own-artifact.
- **prevented vs detected delta.** Add `set_tool_guard` to `LangGraphAdapter`, run the same corpus both ways. Interesting engineering; the *finding* would be a field-level critique, which §22 rules out. Keep as an engineering improvement, do not frame it as a research result.
- **Policy approval / signing gate.** `policy.yaml` already has an unused `metadata` key and every load computes a SHA-256 `policy_hash`. An approval block plus "refuse to enforce if the current hash does not match the last approved hash" is small and demoable. Superseded by §26 as the primary governance answer, but a reasonable secondary mechanism.
- **detguard lint / DCI.** Retired. Mention as a concept in future work only. The formal-model observations (S1/S2/S3 in §4) remain true and citable as *system properties*; what was retired is the claim that they constitute a validated general taxonomy with a new metric.

30. Recommended shape

Contribution. An enforcement-oriented tool-role vocabulary plus mechanical derivation of policy from it — the enforcement counterpart to Stein's monitoring taxonomy.

Validation 1 — instrument. Does the vocabulary cover external tools? (§27.1)

Validation 2 — mechanism. Does derived policy actually enforce? Scoped case study on the three banking deployments, carrying the existence-proof findings (§20, §16) and the harness invariants (§27.3), with the control/treatment design (§13) as the methodological backbone.

Motivation. Cited to UK AISI (§25), not to this project's own corpus.

Engineering deliverable. §28.1 as the concrete improvement; §28.2 if time allows and with failure declared acceptable in advance.

Claims explicitly not made: no prevalence, no field-level critique, no self-referential evasion measurement.

31. Open questions

1. What is the real per-server cost of extracting tool schemas from MCP repos at 50+ scale? (blocks §27.1)
2. Is a second classifier available for inter-rater agreement on the role classification? Without one, §27.1 is single-annotator and must say so.
3. Should the five unused roles (§14) be exercised by extending the banking apps with synthetic tools, or is it more honest to report them as unexercised?
4. Does the seminar's India-regulatory material connect to §26's "standardise the taxonomy, not the policy" argument, or should the two submissions stay separate?